SIMATS ENGINEERING

CO2_AT3 Error Analysis Task

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 20%

Sl. No	Questions	Bloom's Level
1	A tester designs test cases for an online shopping system using equivalence partitioning but incorrectly groups input ranges (e.g., valid and invalid inputs mixed in one class). Additionally, boundary values are not considered. Analyze the given test design and identify all errors in equivalence class partitioning and boundary value analysis. Examine the root causes of these mistakes with proper reasoning. Propose corrected test cases with proper classification and boundary coverage. Justify how your corrections improve defect detection effectiveness.	3
2	A QA engineer performs random testing on a banking application and claims high coverage without using any structured techniques. Critical defects in transaction limits and boundary conditions are missed. Evaluate the testing approach and identify errors in strategy and coverage assumptions. Analyze the root causes behind missing critical defects. Suggest appropriate black box testing techniques to correct the approach. Justify how these techniques ensure better test adequacy and reliability.	3
3	A developer performs white box testing but creates an incomplete control flow graph, missing decision nodes and loops, resulting in poor path coverage. Analyze the errors in the control flow graph and path testing approach. Identify the root causes for inadequate test coverage. Propose corrections by constructing a proper control flow representation and identifying all independent paths. Justify how improved path testing ensures better coverage and defect detection.	5

4	A testing team reports 100% statement coverage for a module but fails to detect logical errors in conditional statements. Evaluate the adequacy of the testing performed and identify gaps in coverage criteria. Analyze why statement coverage alone was insufficient. Recommend improved white box testing strategies such as branch or path coverage. Justify how the revised approach enhances overall test effectiveness and error detection.	5
5	A tester uses Equivalence Class Partitioning for an age-validation field (0-100) but defines only two classes: Negative numbers and Positive numbers. Analyze the errors in this partitioning strategy. Identify why this fails to account for valid vs. invalid age limits and the absence of Boundary Value Analysis. Propose a corrected set of partitions and justify how they ensure more robust input validation.	3
6	A developer claims full Condition Coverage for a complex `if(A && B	
8	During unit testing of a looping structure, a tester checks only the first and last iterations (n=1 and n=max). Defects occurring at n=0 and n=max+1 are missed. Evaluate the errors in this Loop Testing strategy. Analyze the root causes behind these omissions. Recommend a corrected approach for testing simple and nested loops, and justify how it ensures path integrity.	5
9	An engineer utilizes Error Guessing as the primary testing technique for a safety-critical braking system, ignoring structured black box methods. Identify the errors in relying solely on ad-hoc techniques for high-risk modules. Analyze the impact on test adequacy. Suggest a combination of BVA and State Transition Testing to fix the strategy, justifying the increased safety assurance.	5
10	A tester designs a Control Flow Graph (CFG) for a module but fails to represent the "else" branches of multiple nested if statements, leading to 100% statement coverage but 0% branch coverage on error paths. Analyze the errors in graph construction. Identify the root causes for the missing nodes. Propose a corrected CFG and independent paths to ensure branch adequacy.	5

ANSWERS

Question 1: Equivalence Class Partitioning and Boundary Value Analysis in Online Shopping

1. Errors in the given test design

Valid and invalid inputs have been mixed in the same equivalence class. Inputs in one class should be expected to behave similarly; mixing them can hide different validation outcomes.

The input domain and its limits have not been defined. Without specifying an input such as quantity, price, or discount range, the tester cannot classify inputs correctly.

The test set may contain many ordinary values but still miss invalid data, off-by-one errors, and incorrect comparison operators such as < instead of <=.

2. Root causes

The likely causes are incomplete requirement analysis, incorrect understanding of equivalence classes, and a focus on typical values rather than limit values. Equivalence Partitioning (EP) reduces the number of tests by selecting representatives from distinct classes; it does not mean placing all values into one broad class. Boundary Value Analysis (BVA) complements EP by testing the edges of valid and invalid ranges.

3. Corrected classification and test cases

Illustrative requirement used for this answer: an online-shopping quantity field accepts whole numbers from 1 through 10. If the actual assessment system uses a different field or range, replace these example limits with the stated requirements.

Class	Input range / example	Expected result	Representative test
Invalid – below minimum	Quantity ≤ 0	Reject with validation message	0
Valid	Whole number 1–10	Accept quantity	5
Invalid – above maximum	Quantity ≥ 11	Reject with validation message	11
Invalid – non-integer / non-numeric	2.5, “abc”, blank, if not allowed	Reject or require correction	“abc”

BVA point	Test input	Expected result
Minimum − 1	0	Reject
Minimum	1	Accept
Minimum + 1	2	Accept
Typical valid value	5	Accept
Maximum − 1	9	Accept
Maximum	10	Accept
Maximum + 1	11	Reject

4. Why the correction is better

The corrected tests represent each distinct behaviour class and check both sides of the permitted range. They can expose incorrect limits, missing validation, wrong data-type handling, and off-by-one defects with a small, systematic test set. EP and BVA should be used together rather than treated as alternatives.

Question 2: Random Testing in a Banking Application

2. Errors in the approach

- Random testing is being treated as proof of high coverage, even though random inputs do not guarantee that critical requirements or branches have been exercised.
- Transaction limits, zero-value transactions, insufficient funds, and values just below/at/above limits have not been deliberately tested.
- There is no traceability between requirements, test cases, expected results, and recorded coverage.
- A large number of random tests can repeatedly hit common values while rarely hitting rare but high-risk conditions.

2. Root causes

The strategy lacks risk-based planning and boundary-focused design. Random testing is useful for exploring unexpected combinations, but it is weak as the only technique because it does not systematically target equivalence classes, limits, or business rules. Coverage must be measured using defined criteria and evidence, not inferred from the number of tests executed.

3. Recommended black-box techniques and sample tests

Assume, for illustration only, that a single transaction must be between ₹100 and ₹50,000 and cannot exceed the available balance. Actual banking requirements must determine the real limits.

Technique	Example test data / scenario	Expected result / purpose
Equivalence Partitioning	₹500; ₹0; ₹50,001; text input	Accept valid amount; reject invalid amount/type
Boundary Value Analysis	₹99, ₹100, ₹101, ₹49,999, ₹50,000, ₹50,001	Check minimum and maximum limit handling
Decision Table Testing	Amount valid/invalid × sufficient/insufficient balance	Verify every rule combination
State Transition Testing	Initiated → authenticated → approved/declined → posted/reversed	Check valid and invalid transaction state changes
Error Guessing (supplementary)	Duplicate submit, timeout, retry, network loss	Check duplicate debit and recovery behaviour
Random / fuzz testing (supplementary)	Unexpected amounts, formats, sequences	Discover unanticipated inputs after core coverage

4. Why this improves adequacy and reliability

EP gives representative coverage of valid and invalid input groups; BVA targets limit defects; decision tables cover combinations of conditions; and state-transition tests validate transaction lifecycle behaviour. Risk-based prioritisation and requirement-to-test traceability make coverage measurable. Random testing remains useful as an additional exploratory technique.

Question 3: Correcting an Incomplete Control Flow Graph and Path Testing

1. Errors and root causes

- Decision nodes and their true/false edges are missing, so the graph does not represent every possible control transfer.
- Loop entry, loop body, loop-back edge, and loop-exit edge are omitted or incorrectly represented.
- The resulting graph hides feasible execution paths, making reported path coverage unreliable.
- Likely causes include translating source code into a graph too quickly, overlooking compound conditions, and not validating the graph against every branch in the code.

2. Corrected illustrative control-flow example

Because the question does not provide source code, the following small example is used to show the correct method. It includes an input decision and a loop.

Pseudocode: read n; if n < 0 then print "Invalid" else set i = 0; while i < n do print i; i = i + 1; print "Done".

Node	Operation	Outgoing edge(s)
1	Start / read n	→ 2
2	Decision: n < 0?	True → 3; False → 4
3	Print "Invalid"	→ 8
4	Initialise i = 0	→ 5
5	Loop decision: i < n?	True → 6; False → 7
6	Print i; increment i	→ 5 (loop-back)
7	Print "Done"	→ 8
8	End	—

3. Independent paths and test cases

The cyclomatic complexity for this connected graph is V(G) = E − N + 2P = 9 − 8 + 2(1) = 3. Therefore, three independent paths form a basis for this graph; this does not mean every possible loop repetition is covered.

Path	Representative input	Path exercised
P1	n = -1	1 → 2(True) → 3 → 8
P2	n = 0	1 → 2(False) → 4 → 5(False) → 7 → 8
P3	n = 2	1 → 2(False) → 4 → 5(True) → 6 → 5(True) → 6 → 5(False) → 7 → 8

For loop robustness, also test n = 1 and a larger permitted value, and test invalid or extreme inputs according to the specification. Path testing should focus on feasible paths; exhaustive testing of all loop counts is usually impossible.

4. Justification

A complete graph makes each decision outcome and loop transition visible. Independent-path tests exercise linearly independent control-flow behaviour, improving detection of incorrect conditions, missing loop exits, and faulty loop updates.

Question 4: Why 100% Statement Coverage Is Not Enough

1. Evaluation and coverage gaps

Statement coverage measures whether each executable statement has run at least once. It does not guarantee that every decision outcome has been tested or that a compound condition behaves correctly. A test can execute all statements while taking only the true outcome of a decision.

2. Example

Code: if (balance >= amount) { approve(); } else { decline(); }

Test	balance	amount	Executed outcome
T1	100	50	True → approve()
T2	200	100	True → approve()

The example above does not cover the false branch. If the test suite has 100% statement coverage in a different module, the same principle still applies: statements can be executed without exercising both outcomes of every decision. For this example, add balance = 40 and amount = 50 to exercise decline().

3. Improved strategy

- Branch/decision coverage: execute both true and false outcomes of each decision at least once.
- Condition coverage: make each atomic Boolean condition evaluate to both true and false, where applicable.
- Modified Condition/Decision Coverage (MC/DC): for safety-critical logic, demonstrate that each condition can independently affect the overall decision outcome.
- Basis-path testing: use the control-flow graph and cyclomatic complexity to identify a set of independent paths.

4. Justification

Branch coverage detects missing or incorrectly implemented alternatives that statement coverage can miss. Condition and MC/DC testing are particularly useful for compound Boolean expressions. Path testing adds control-flow insight, although full path coverage may be infeasible when loops create infinitely many paths. Coverage metrics support, but do not replace, assertions, requirement-based tests, and code review.

Question 5: Correct Equivalence Classes and Boundaries for Age 0–100

1. Errors in the original partitioning

- “Negative” and “positive” are not sufficient classes: positive values include both valid ages (1–100) and invalid values above 100.
- Zero is not positive, but it is valid under the stated range 0–100; it is therefore not correctly handled by those two classes.
- The partitioning does not explicitly identify non-numeric values, decimals, blanks, or missing input if the field requires an integer.
- No boundary tests are specified at 0 and 100 or immediately outside the valid range.

2. Corrected equivalence classes

Class	Input examples	Expected result
Invalid – below minimum	-1, -10	Reject
Valid – lower boundary	0	Accept
Valid – interior values	1, 25, 50, 99	Accept
Valid – upper boundary	100	Accept
Invalid – above maximum	101, 120	Reject
Invalid – wrong format	“abc”, 2.5, blank, if integer required	Reject / request valid input

3. Boundary Value Analysis tests

Test value	Reason	Expected result
-1	Minimum − 1	Reject
0	Minimum	Accept
1	Minimum + 1	Accept
50	Representative middle value	Accept
99	Maximum − 1	Accept
100	Maximum	Accept
101	Maximum + 1	Reject

These classes distinguish valid from invalid behaviour and cover the exact limits and adjacent values. This improves detection of range-check mistakes and incorrect handling of zero or the maximum age.

Question 6: Condition Coverage for a Compound Boolean Decision

1. Key concept and likely error

For A && B, the decision is true only when both A and B are true. Condition coverage requires each atomic condition to take both true and false values across the test suite. Because the language may use short-circuit evaluation, B is not evaluated when A is false. Therefore, a test that makes A false does not by itself prove that B has been evaluated as false.

2. Correct test design

Test	A	B value / evaluation	Decision A && B	Purpose
T1	True	True (evaluated)	True	Both conditions true
T2	True	False (evaluated)	False	B becomes false while A is true
T3	False	Not evaluated under short-circuit rules	False	A becomes false; note B is not evaluated

T1 and T2 show both outcomes for B and true for A; T3 makes A false. Under short-circuit semantics, B cannot be evaluated on T3, so report the coverage tool's precise definition and do not claim that B was evaluated in that test. If the assessment expects non-short-circuit evaluation, follow the language and instrumentation specified by the question.

3. Coverage and justification

Decision coverage requires the overall expression to be both true and false. Condition coverage requires each atomic condition to be true and false when evaluated. They are different criteria; neither alone guarantees all possible paths. For safety-critical compound logic, MC/DC may be appropriate because it checks whether each condition independently changes the decision outcome.

Question 7: Decision Table Testing for a Discount System

1. Errors in the incomplete table

- Not all combinations of Premium Member and Coupon Applied have been tested.
- The table may omit a rule, duplicate a rule, or assign more than one outcome to the same combination.
- Discount stacking and eligibility rules have not been stated clearly, so developers and testers may implement different interpretations.

2. Revised exhaustive decision table

Illustrative policy assumption: premium membership gives 10% off, a valid coupon gives 5% off, and both discounts stack additively for 15% total. If the actual business rule differs, use the approved policy and update the expected results.

Rule	Premium member?	Valid coupon applied?	Expected discount	Action
R1	No	No	0%	Charge full price
R2	No	Yes	5%	Apply coupon discount
R3	Yes	No	10%	Apply premium discount
R4	Yes	Yes	15%	Apply both discounts

3. Example calculation tests

Base price	Premium?	Coupon?	Expected final price*
₹1,000	No	No	₹1,000
₹1,000	No	Yes	₹950
₹1,000	Yes	No	₹900
₹1,000	Yes	Yes	₹850

*Calculated using the illustrative additive policy above, before taxes, shipping, or rounding rules. Those rules should be tested separately if applicable.

4. Why it is effective

Two Boolean conditions create four combinations, and the revised table includes all four. Each rule has one unambiguous outcome, allowing the tester to verify every combination and catch missed discounts, double discounts, or incorrect precedence. Add tests for expired, invalid, or ineligible coupons if those states exist in the requirements.

Question 8: Loop Testing: Zero, Boundary, and Nested Iterations

1. Errors and causes

- Testing only n = 1 and n = max misses zero iterations, which may reveal incorrect loop-entry behaviour.
- n = max + 1 is not tested, so the system's response to an out-of-range iteration count may be unknown.
- Testing only the first and last iteration can miss defects in the second iteration, loop termination, counter updates, and repeated execution.
- For nested loops, the interaction between inner and outer loops can produce defects that a single-loop test set will not reveal.

2. Correct strategy for a simple loop

Let max be the maximum permitted iteration count in the requirements. Do not execute max + 1 as a valid workload if it could be unsafe; test it as an input-validation case and expect rejection or safe handling.

Test case	Iteration count	Expected purpose
L1	n = 0	Loop body executes zero times; verify correct skip/exit
L2	n = 1	Minimum positive loop execution
L3	n = 2	Detect early repeat/counter defects
L4	Typical n	Verify normal repeated execution
L5	n = max − 1	Near upper limit
L6	n = max	Maximum permitted workload
L7	n = max + 1	Input must be rejected or safely handled

3. Nested-loop testing

Start with the inner loop at its minimum practical values while testing the outer loop at its minimum, typical, and maximum values.

Increase the inner loop through zero, one, two, typical, and maximum iterations while holding the outer loop at a small value.

Then test combinations such as outer = 0, inner = 0; outer = 1 with inner = 0/1/max; and outer = max with a small inner count.

Check loop initialization, termination conditions, counter updates, break/continue behaviour, and resource limits. Use risk-based selection because exhaustive combinations can grow rapidly.

4. Justification

The corrected strategy checks entry, single execution, repetition, termination, and input limits. For nested loops it also examines interactions between loop levels. This improves confidence in loop control and path integrity without claiming that a finite test set covers every possible iteration count.

Question 9: Safety-Critical Braking System: Error Guessing, BVA, and State Transitions

1. Problems with relying only on Error Guessing

Error Guessing depends heavily on tester experience and may miss uncommon but hazardous combinations. It provides no systematic guarantee that requirement limits, valid states, invalid transitions, and recovery behaviour are covered.

Without documented expected results and traceability, test adequacy and repeatability are difficult to demonstrate.

.

2. Combine structured methods

Use the braking-system specification to define the actual pressure, speed, temperature, timing, and sensor limits. Do not invent safety thresholds; the following table uses symbolic limits L and U.

Technique	Test scenario	Expected result
BVA	Sensor value just below L, at L, just above L	Reject, accept, or trigger specified safe response at the correct lower limit
BVA	Sensor value just below U, at U, just above U	Verify upper-limit handling and no off-by-one error
BVA	Zero, maximum representable value, missing/corrupt sensor input	Safe, specified handling without uncontrolled braking behaviour
State transition	OFF/STANDBY → ACTIVE when activation conditions are met	Enter active state only when permitted
State transition	ACTIVE → BRAKING on a valid brake request or hazard signal	Apply the specified braking response
State transition	ACTIVE → FAULT on sensor failure; FAULT → SAFE/RECOVERY according to requirements	Enter a safe state and recover only through permitted transition
State transition	Invalid transition request, repeated signal, or signal loss	Reject or safely handle the invalid transition

3. Recommended assurance process

- Derive tests from safety requirements and hazard analysis; prioritise high-severity failure modes.
- Use BVA for all numeric safety thresholds and State Transition Testing for operating modes, faults, recovery, and invalid event sequences.
- Retain Error Guessing as a supplementary technique for expert-led scenarios such as sensor glitches, timing races, and repeated signals.
- Record expected results, actual results, requirement links, test evidence, and regression outcomes. Apply independent review and the safety standard required by the project.

4. Justification

BVA systematically exercises limit behaviour, while state-transition tests cover legal and illegal changes between operating modes. Together they improve repeatability and hazard coverage. They do not, by themselves, prove the system safe; safety analysis, verification, validation, and applicable regulatory processes are also required.

Question 10: Correcting a CFG with Missing Else Branches

1. Errors and root causes

- The graph omits false edges from nested decisions, so error-handling and alternative paths are not represented.
- A statement-coverage result can be misleading if tests execute statements on the successful path but never test false outcomes of each decision.
- "0% branch coverage on error paths" indicates that the false/error edges were not exercised; it is not evidence of adequate decision testing.
- Likely causes include drawing the CFG from only the happy path, omitting implicit else paths, and failing to reconcile the graph with the source code.

2. Independent paths and tests

Path	inputValid	authorised (evaluated only if inputValid)	Expected behaviour
P1	True	True	processRequest executes
P2	True	False	showAccessError executes; request is not processed
P3	False	Not evaluated	showInputError executes; authorisation check is skipped

The graph has three decision outcomes/independent basis paths for this simple structure. With E = 7 edges, N = 6 nodes, and P = 1 connected component, V(G) = E − N + 2P = 7 − 6 + 2 = 3. The exact count depends on the node/edge convention used, but the three test paths above exercise the three feasible outcomes.

3. Verification and justification

Check that every if statement has true and false edges and that each branch leads to the correct next node or termination.

Execute P1, P2, and P3 and verify both the expected output and that forbidden actions do not occur.

Use branch coverage to confirm each decision outcome was exercised; use condition coverage or MC/DC for compound conditions when required.

Compare the CFG directly with the source code and update tests when the implementation or requirements change.

A complete CFG exposes the error paths and enables targeted tests that statement coverage alone may miss. The corrected approach improves branch adequacy and helps detect missing validation, incorrect error handling, and accidental processing of invalid or unauthorised requests.

Code of Conduct:

*I **BHUVANESHWARI B A(192421195)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: BHUVANESHWARI B A

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand